

Better Manipulation for Robot Agents

System 0 Actions and Grounded Force–Position Control

Research Proposal I · Zimo Wen

September 2026

Abstract

Robot-RSI commonly expresses a manipulation goal as an end-effector target that an inverse-kinematics and trajectory backend converts into robot motion. This proposal asks when better manipulation requires a different action interface: a fast local policy that directly predicts robot actions, an explicit statement of desired interaction passed to a fixed feedback controller, or a learned local force policy conditioned on that statement. The research is grounded in specific contact mechanics rather than assuming that lower-level output is always superior. Force Policy already predicts interaction frames, force-control masks, reference wrenches, and motion chunks; these are prior capabilities, not proposed inventions. The intended contribution is a task-grounded interface that connects an agent’s interpretation of the scene to executable interaction requirements, with matched comparisons against direct-action and existing force-policy baselines. Success, contact quality, generalization, and cost jointly determine whether manipulation improves.

1. Problem Definition and Evidence from Existing Systems

What System 0 means here. We use System 0 operationally for a fast local sensorimotor policy below task reasoning, not as the name of a unique checkpoint or a claim that every such policy outputs torque. Its output might be joint position, joint velocity, EEF increments, or torque. These interfaces must be distinguished. ACT explicitly predicts future target joint positions [1]; Reactive Diffusion Policy uses a slow latent policy and a fast tactile-conditioned decoder [2]. Neither observation alone establishes arbitrary joint-torque control.

The current baseline. In the inspected Robot-RSI LIBERO path, EEF targets are passed through pose solving and servo execution to joint increments. This is a concrete target-plus-IK baseline. A direct joint-action policy can avoid a separate target-to-joint IK solve at its own output boundary, but it still needs a robot servo, limits, and dynamics. Conversely, predicting an EEF delta can still invoke IK or a Jacobian-based controller internally. Eliminating a named IK call is not itself a manipulation result.

What Force Policy already solves. Fang et al.’s Force Policy [3] predicts an interaction frame, a six-axis selection mask, reference wrench, and motion chunk. Its global vision and local force policies are asynchronously scheduled. The paper also uses task semantics and a VLM in demonstration-based interaction-frame recovery. Therefore, neither adding desired force to an output nor using language to help determine interaction structure can be claimed as new by itself.

A more specific hypothesis. An agent can improve manipulation by selecting and updating *scene-grounded interaction requirements*: which motion is intended, which contact should be maintained, in what frame, and within which task-supported force range. These requirements can drive fixed control or condition a learned local policy. The hypothesis is only supported if this improves over existing Force Policy and appropriately tuned controllers under matched information and data. Otherwise the result may be a useful action-interface comparison rather than a new control method.

Existing integration is not an experiment. A separate Robot-RSI force-control integration documents official ForcePolicy adapters, processors, a scheduler bridge, and hybrid-control streaming. Its recorded validation used fake SDKs and substitutes; it did not establish real checkpoint inference or hardware contact performance. This is more infrastructure than contact-stop alone, but it remains an implementation starting point rather than evidence for the proposed paper.

2. Approach A: Direct System 0 Action Prediction

Input and output. A slow agent supplies an instruction or bounded task-phase representation z_t . A local policy receives z_t , current images or visual features, proprioception, and a short sensor history. The primary direct-action variant predicts joint-position or velocity sequences. A position target can be executed by a fixed servo, illustratively $\tau = K_p(q^{\text{ref}} - q) - K_d\dot{q} + \tau_{\text{ff}}$, where the servo gains and feedforward terms belong to the declared backend. The language model does not generate each servo-cycle torque token.

What is being learned. Train the local mapping from the same physical demonstrations used by the EEF-target comparison, retaining actual commanded joint targets rather than substituting measured positions indiscriminately. A direct policy may learn contact-sensitive corrections and avoid poor target-to-joint choices, but it can also overfit embodiment and workspace geometry. Evaluate both ordinary manipulation and configurations where the existing pose solver struggles. Report solver failures separately from failures of the policy’s intended behavior.

Additional variants. Compare direct joint commands with a local EEF-increment head and, only when a torque interface and training setup are validated, a bounded torque or residual head. The EEF head is not labeled IK-free if its backend performs pose inversion. Keep sensing, action horizon, training data, and servo limits matched. The question is whether changing the executable interface improves success and contact behavior, not whether one interface is syntactically closer to a motor.

3. Approach B: Interaction Requirements Realized by Fixed Control

A grounded interaction contract. The agent emits $c_t = (\mathcal{F}_t, S_t, \mathcal{W}_t^{\text{ref}}, \xi_t^{\text{motion}}, \mathcal{B}_t, \mathcal{G}_t)$: an interaction frame, force/position axis mask, reference wrench, desired motion, calibrated bounds, and phase/termination conditions. The contract records the source of each physical quantity, such as a calibrated tool profile, measured demonstration, visible geometry estimate, or bounded development experiment. A phrase like “press gently” does not uniquely determine a force in newtons.

What the fixed program does. The program converts frames, validates units and signs, selects compatible motion and force axes, and runs a feedback controller. For a simple admittance realization, an illustrative velocity reference in one common frame is

$$\xi^{\text{cmd}} = (I - S)\xi^{\text{motion}} + SA(\mathcal{W}^{\text{ref}} - \mathcal{W}^{\text{meas}}),$$

with dimensioned translational/rotational admittance A . A vendor hybrid controller can instead implement this behavior internally. Robot dynamics, contact, saturation, and the motion backend still matter. There is no unique algebraic inversion from a desired force to a joint configuration; force is achieved through closed-loop interaction.

Physical interpretation. A desired wrench is different from measured wrench, a maximum-force stop condition, and impedance parameters. Express force and moment in a specified frame and at a specified origin; moving the wrench origin requires the appropriate moment transformation. Document whether the SDK represents force felt by the TCP or force exerted on the environment. Stiffness and damping may be fixed controller settings or additional declared outputs, but they are not interchangeable with the force-axis mask used by Force Policy.

Uncertainty. If geometry, force provenance, or the relevant task phase is uncertain, the agent must request an allowed observation or use a declared conservative calibration procedure. It must not invent a target force. All development identification and calibration costs are included when comparing with fixed or manually specified contracts.

4. Approach C: A Contract-Conditioned Local Force Policy

Method. Condition a learned local policy on the grounded contract, wrist observations, and force and proprioceptive history. It updates the interaction frame, mask, reference wrench, and local motion within declared task bounds. The motivation is that the agent specifies the intended physical outcome while the fast policy adapts to local contact. This reuses the established Force Policy representation; the research variable is the value and reliability of the explicit task-grounded conditioning and its updates.

Required comparison. Compare the official Force Policy architecture and labels with an otherwise matched contract-conditioned version, including a hand-specified-contract upper reference and a no-contract ablation. Preserve its processor, frame conventions, and task-specific labeling choices where applicable. Do not attribute gains from extra demonstrations, a different scheduler, or a stronger vision backbone to the contract. The authors report a 50 Hz local policy and unified scheduling; those are source-specific measurements, not achieved frequencies for our integration.

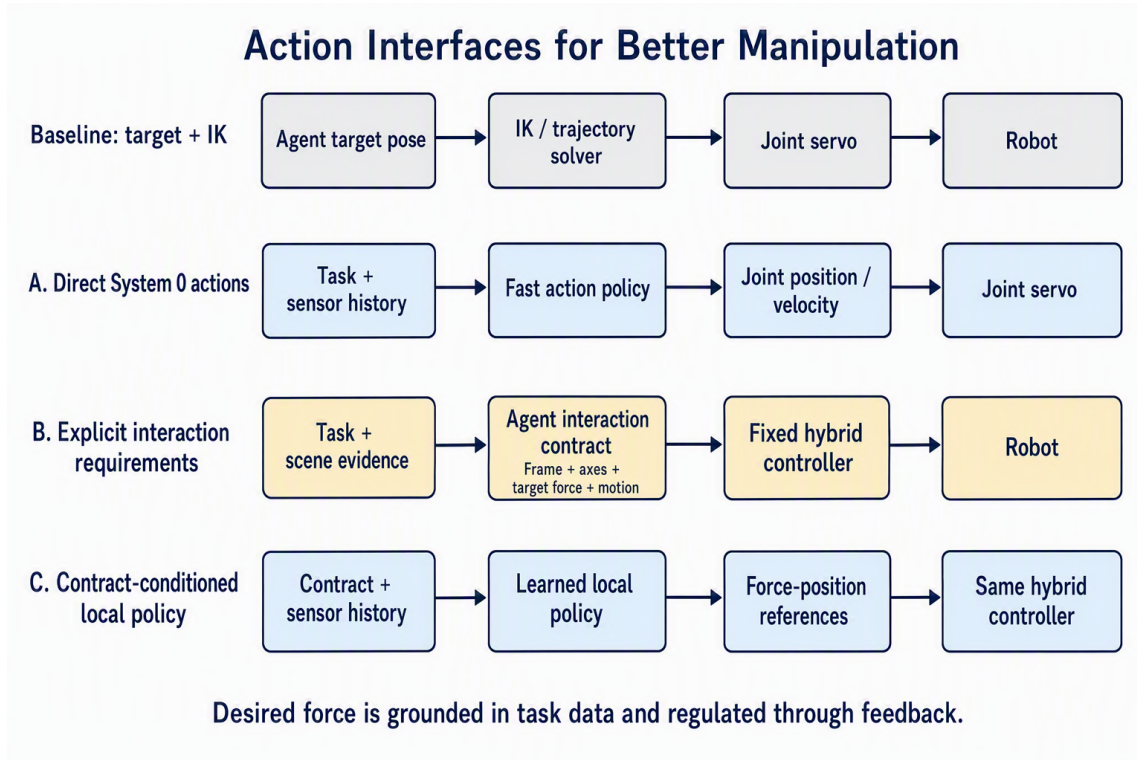


Figure 1: Target-plus-IK is the reference. A predicts direct local actions; B grounds interaction requirements for fixed feedback control; C conditions a learned local policy on those requirements. GPT-image-2 conceptual figure. Each backend and actual command space must be documented.

Why an agent could add value. Test task-phase or instruction changes that alter the desired interaction while preserving object appearance: maintaining contact versus withdrawing, or surface following versus a position-only approach. A contract can make this distinction explicit, but existing learned policies may already infer it. Only matched experiments on held-out task–geometry combinations can establish the agent’s additional contribution.

5. Concrete Scenarios and Experimental Design

Scenario	Appropriate interaction requirements	What must not be assumed
Surface following with a tool or pad	Track tangential motion, regulate normal contact, and specify tool orientation and force provenance.	The same normal force works for different surfaces, materials, or tools.
Constrained insertion	Advance along the identified insertion axis; permit declared lateral/angular compliance and cap axial resistance.	Higher insertion resistance should always be answered by larger constant pushing force.
Drawer or articulated sliding	Follow the estimated motion constraint; use lateral compliance and resistance/position feedback.	Drawer resistance is a universal desired axial-force target rather than a changing observation or limit.

Choosing force references. For surface following, derive an admissible normal-force range from development demonstrations and tool/material calibration, then select or predict a reference within that range. Report the task effect, such as sustained contact or achieved surface coverage, together with material-specific upper limits. For insertion, start with a declared low-speed motion and resistance limit; zero lateral wrench targets or an impedance policy may permit alignment only when the actual controller supports those semantics. Desired force is used only when the task truly requires it. No generic numeric force values are proposed as transferable robot settings.

Initial panel. Use four held-out object/geometry configurations in each of the three task families, three levels of relevant physical variation, and five initial seeds: 180 task episodes per method. Development and validation use distinct objects and task–geometry combinations. Test single changes before combinations: surface tilt, compliance, friction, insertion clearance, or articulated-axis error. Verify that the task remains physically solvable. These contact cells are declared extensions; standard LIBERO scores are reported separately.

Baselines. Include the current target-plus-IK path with tuned feedback; direct joint-action and local EEF-action policies; manually specified fixed hybrid control; the published Force Policy method where data and interfaces support it; agent-generated contracts with fixed control; and contract-conditioned local learning. Reactive Diffusion Policy provides an additional slow–fast sensorimotor comparison where its data requirements can be matched. Record the exact output space of every method instead of classifying all learned actions as torque control.

Data and hardware realism. Use synchronized images, commanded and measured joint/EEF motion, and calibrated wrench history. Force targets and interaction-frame labels need appropriate processing; a force sensor stream is not automatically an optimal reference trajectory. Force Policy’s public Push and Flip checkpoint is not a model for arbitrary insertion, scraping, or cutting. Obtain task-matched training data for comparisons. Hardware evaluation requires validated tools, frames, sensing, and control interfaces; no robot execution is part of preparing this proposal.

Standard task reporting. On the separate 120-task LIBERO short panel, report frozen task-level pass@10 with spatial/object/goal and task/object/swap/lan breakdowns. Keep original benchmarks, contact extensions, historical adaptive results, and trained-policy results distinct.

6. Planned Ablations and Evidence for Better Manipulation

Experiment A: action space versus controller quality. Compare target-plus-IK, direct joint actions, and local EEf actions using the same demonstrations, sensing, and comparable model capacity. Tune the IK baseline and disclose the trajectory and servo settings. Evaluate task completion, contact disturbances, and motion quality, including held-out configurations. A direct policy beating an untuned solver does not establish a general action-representation advantage.

Experiment B: desired contact versus force limits. Compare position control with force sensing, contact-stop behavior, fixed hybrid references, and task-grounded references. All methods observe the same wrench signal. Separately ablate frame estimation, axis selection, desired wrench, and motion intent. This distinguishes the value of force feedback from the value of an explicit target. A clamp that prevents large forces is not evidence that the intended force was tracked.

Experiment C: what the agent contributes beyond Force Policy. Compare no contract, fixed human-specified contracts, generated contracts, and generated contracts updated after observable contact changes. Match training labels, local network, and scheduler. Test instruction/phase changes and new task–geometry combinations while holding the scene appearance approximately constant where possible. Add incorrect-frame, incorrect-mask, and unsupported-reference diagnostics only offline in simulation and label them clearly. If existing Force Policy matches performance, limit the novelty claim to the measured task interface or generalization setting.

Experiment D: grounding and calibration. Compare references obtained from task-matched demonstrations, calibrated ranges, and language-only numeric proposals as a simulation diagnostic. The last condition tests the cost of ungrounded specification, not an endorsed deployment strategy. Measure whether the proposed contract predicts a useful physical quantity and whether its provenance supports transfer to a new object. Calibration trials and extra sensing are counted as resources.

Metrics. Final simulator/harness success is primary. During defined contact phases, measure reference-force RMSE, contact-loss duration, peak force, impulse, and task-specific physical effects. Report pose error and settling behavior in motion phases. Force tracking alone can be perfect for a useless reference, so it must be paired with actual manipulation success. An insertion that reaches high force without insertion progress is a failure, not successful force regulation.

Observed pattern	Supported interpretation
Direct actions help pose-solver-sensitive cases only	A useful embodiment/interface boundary, not universal superiority of lower-level control.
Grounded references improve contact and task success	Supports task-specific interaction specification, after matching feedback access.
Force tracking improves but task success does not	The chosen physical objective may be inappropriate.
Contract conditioning matches unmodified Force Policy	No incremental learned-control contribution has been established.
Manual contracts work but generated ones fail	Grounding the agent’s physical specification is the bottleneck.

Use paired seeds and task-clustered intervals, with multiple training seeds for learned variants. Report inference, calibration, training, and execution costs separately. Additional samples are determined from validation variance; the proposed panel is not an assertion of sufficient statistical power.

7. Implementation Boundaries and Expected Contribution

Reuse before invention. The inspected Force Policy code and paper already expose action/frame/wrench/mask outputs, task-specific processors, and a slow-fast scheduler. The Robot-RSI integration documents frame conversion and fixed SDK realization. Reuse these interfaces as declared baselines rather than rebrand them. The new paper must identify a measured gain from task-grounded specification, direct local action learning, or their combination; these are hypotheses, not established capabilities.

Control facts that must remain explicit. Joint targets still require a servo. EEF targets may still require IK. A desired wrench needs a contact model or feedback realization, not a unique inverse solution. Impedance sets a force-motion relationship, while hybrid control selects which axes regulate motion or force. Wrench frame, origin, sign, units, and sensor compensation are part of the method. SDK command acceptance, completed streaming duration, or a fake-interface test does not establish contact convergence or task success.

Scope of the agent and the local policy. The agent selects task phases and grounds interaction requirements on a slower timescale. The local policy or fixed feedback program performs sensor-conditioned execution. Actual update rates and delays must be measured. We do not claim that GPT emits reliable high-frequency torques. System 0 is the functional placement of the local sensorimotor module, whose action representation and training are specified separately for each variant.

Evidence and failure accounting. Final outcomes come only from the simulator or independent harness. Exclude provider, transport, image, resource, and interrupted records from valid-verdict success denominators and report them separately, with implementation errors and coverage. Retain force curves, videos, rejected specifications, failed runs, and exact manifests. Report tokens, VLM and unmetered calls, wall time, and perception/action/recovery time. Learned models require matched and held-out simulation evaluation; training loss does not establish manipulation improvement.

Expected paper contribution. The strongest plausible result is an empirically supported interface between task reasoning and physical interaction: the agent specifies grounded, scene-appropriate motion and contact requirements, and an explicitly defined local realization executes them better under selected generalization conditions. A useful negative result is that direct actions or published Force Policy already suffice. Either outcome must explain concrete manipulation behavior rather than equating lower abstraction with greater intelligence. The second proposal separately studies generated reactive policy structure; the third retains GPT to audit and correct VLA actions.

References

- [1] Zhao et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT). 2023. <https://arxiv.org/abs/2304.13705>
- [2] Xue et al. Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. 2025. <https://arxiv.org/abs/2503.02881>
- [3] Fang et al. Force Policy: Learning Hybrid Force-Position Control Policy under Interaction Frame for Contact-Rich Manipulation. 2026. <https://arxiv.org/abs/2602.22088>
- [4] Force Policy official source, model heads, processors, and scheduler. <https://github.com/force-policy/ForcePolicy>
- [5] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [6] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>